

Doc. no.	P0914r0
Date:	2018-02-08 at 14:25:00 UTC
Reference:	ISO/IEC TS 22277, C++ Extensions for Coroutines
Audience:	EWG
Reply to:	Gor Nishanov < gorn@microsoft.com >

Add parameter preview to coroutine promise constructor

Issue

Users of C++ coroutines have long been requesting an ability to access coroutine parameters in the constructor of the coroutine promise. Current workaround is to override `operator new` of the coroutine promise that does allow observing of coroutine parameters, then extract the value of the desired parameter and store it in a `thread_local` variable and later extract the value from the `thread_local` in the coroutine promise default constructor and store the value in the coroutine promise.

The workaround used is not reliable as compiler is allowed to elide heap allocation for the coroutine state and elide invocation of `operator new`. Coroutines need direct and reliable way of expressing the desired behavior.

Suggestion is to give a coroutine promise an ability to look at the coroutine parameters.

Before:

```
struct my_promise_type {
    static thread_local cancellation_token saved_tok;
    cancellation_token tok;

    template <typename... Whatever>
    void* operator new(size_t sz, cancellation_token tok, Whatever const&...) {
        // BROKEN: May not get called if heap allocation is elided.
        saved_tok = tok;
        return ::operator new(sz);
    }

    my_promise_type() : tok(saved_tok) {}
    ...
};
```

After:

```
struct my_promise_type {
    cancellation_token tok;

    template <typename... Whatever>
    my_promise_type(cancellation_token tok, Whatever const&) : tok(tok) {}
    ...
};
```

Wording:

[Proposed wording is relative to [N4680](#) (ISO/IEC TS 22277)].

Modify paragraph 8.4.4/3 as follows:

```
{
  P p promise-constructor-argumentsopt;
  co_await p.initial_suspend(); // initial suspend point
  try {
    F
  } catch(...) { p.unhandled_exception(); }
  final_suspend:
  co_await p.final_suspend(); // final suspend point
}
```

where an object denoted as p is the promise object of the coroutine, and its type P is the *promise type* of the coroutine, and $promise-constructor-arguments_{opt}$ is determined as follows: Overload resolution is performed on a promise constructor call created by assembling an argument list with lvalues $p_1 \dots p_n$. If matching constructor is found, then $promise-constructor-arguments_{opt}$ is $(p_1 \dots p_n)$, otherwise $promise-constructor-arguments_{opt}$ is nothing.

Add underlined text to 8.4.4/11:

When a coroutine is invoked, a copy is created for each coroutine parameter. Each such copy is an object with automatic storage duration that is direct-initialized from an lvalue referring to the corresponding parameter if the parameter is an lvalue reference, and from an xvalue referring to it otherwise. A reference to a parameter in the function-body of the coroutine and in the call to coroutine promise constructor is replaced by a reference to its copy.

Implementation experience

Implemented in clang trunk.